Conference Paper Title*

*Note: Sub-titles are not captured for https://ieeexplore.ieee.org and should not be used

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

5th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

6th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—This document is a model and instructions for LaTeX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.**

***Index Terms*—component, formatting, style, styling, insert.**

I. Achieving efficient elasticity

A. In-memory data organization

This section presents the design and implementation of the mechanisms that enable efficient and minimally disruptive elasticity in Aqueduct. We first describe the in-memory data organization, focusing on how Aqueduct's RDMA-friendly memory allocator ensures that logical data groupings (slots) are reflected in the physical memory layout, enabling efficient high-speed data transfers (Section I-A). We then detail the migration protocol, outlining how Aqueduct orchestrates seamless and transparent data migrations across nodes while continuing to serve client requests (Section I-D). A key part of the migration process is the index update, which integrates transferred key-value pairs into the destination's serving data structures (Section I-G).

Building upon this foundation, we improve upon the base migration protocol by overlapping computation and communication for better resource utilization (Section **??**). Finally we introduce a self-regulating mechanism that allows Aqueduct to preserve client workload performance even during intensive migration phases (Section **??**).

Key-value stores (KVSs) typically organize data in logical partitions called *shards*, *slots*, or *tokens* in different systems [1]–[3]. In this paper, we uniformly refer to them as *slots*, regardless of the underlying system.

In most KVSs today, in-memory data organization does not necessarily align with the logical organization of data (i.e., slots). The reason is that general-purpose memory allocators typically disregard the logical affinity between KV pairs, leading to data of the same slot being scattered across different memory locations within a node. Consequently, during data migrations, a node must gather scattered memory buffers to transfer a slot, resulting in long *gather* lists for RDMA operations. Such long RDMA chains of small buffers are inefficient, severely degrading transfer bandwidth and efficiency.

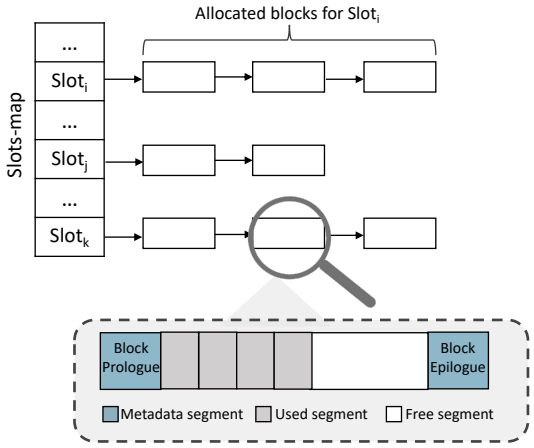

Fig. 1. Allocator internal data organization

To address this, Aqueduct introduces a bulk-transfer-oriented, RDMA-friendly memory allocator that ensures in-memory layouts mirror the logical slot structure. Upon a memory allocation request, Aqueduct's allocator, leveraging the knowledge of which slot ID each key-value pair belongs to, aims to place all data of a slot into contiguous physical memory regions.

The allocator maintains metadata in the form of a hash table, where each bucket represents a distinct slot (Fig. 1). Since it is challenging to predict all records and their memory requirements for each slot without prior knowledge of the

Identify applicable funding agency here. If none, delete this.

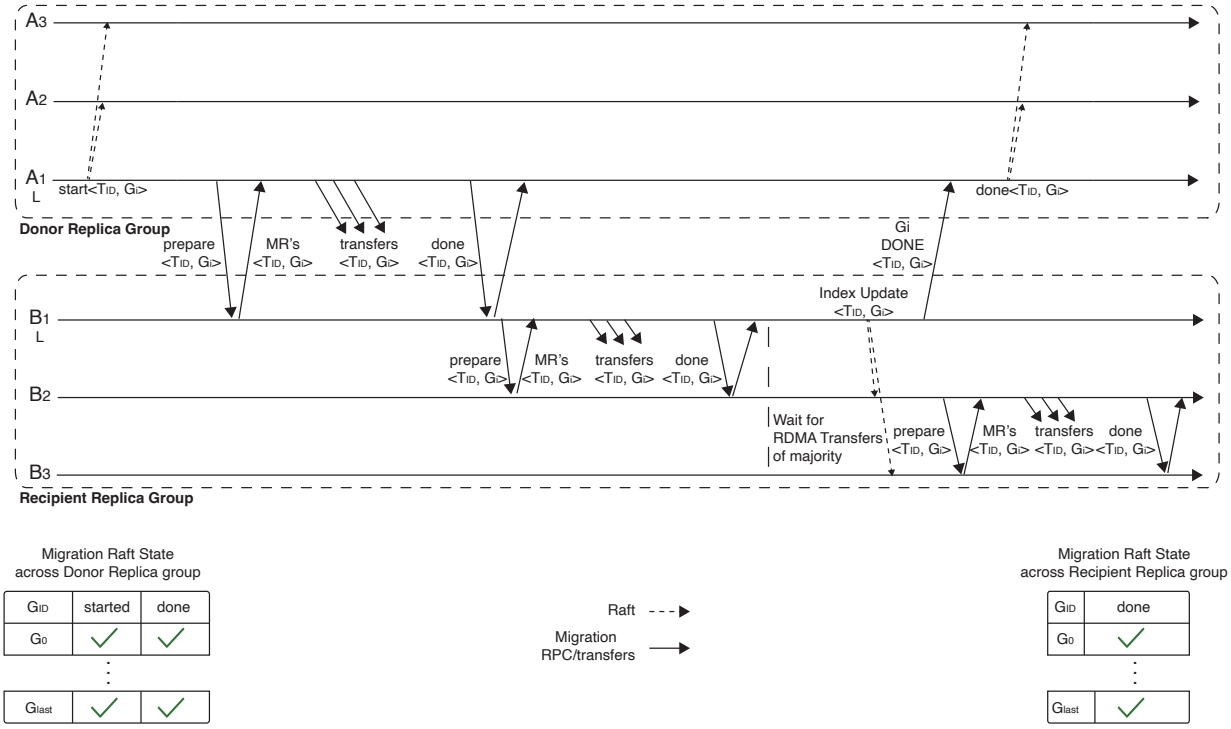

Fig. 2. Aqueduct baseline migration protocol

dataset, the allocator operates using fixed-size memory regions called *blocks*. Records within a slot are packed across one or more blocks. When a block becomes full, a new one is allocated to continue storing the slot's data. Each block includes metadata *prologue* and *epilogue* to efficiently keep track of the block boundaries as well as the used and free segments in it.

We replace the generic memory allocation policy with our tailored allocator. Each memory allocation request is tagged with its associated slot, allowing the allocator to co-locate key-value pairs belonging to the same slot.

The allocator is a full-featured memory manager that supports all standard operations, including inserts, deletes, and updates. To efficiently manage memory reuse, it employs a free list to track available memory regions, enabling fast identification of suitable space for new allocations. Additionally, it supports coalescing adjacent free regions to reduce fragmentation and improve overall space utilization.

For every inserted record, the allocator adds a metadata header that records the segment boundaries and includes padding to ensure proper alignment to double word (8 bytes) boundaries. This allows to use the least significant bits of the memory address (that we assume it is zero due to alignment) to encode addition metadata (e.g. the status of the segment).

During the migration process, when a node offloads certain slots to another node, it can request a list of memory blocks associated with those slots. This slot-aware memory organization substantially reduces the number of descriptors in RDMA operations (i.e., shorter gather lists composed of larger buffers), leading to significantly improved RDMA performance. Furthermore, communicating through fewer, larger memory regions (MRs) optimizes memory *registration* overhead[1]. RDMA-capable NICs (RNICs) can more efficiently manage memory resources when buffers are contiguous, allowing for more efficient use its memory management resources (e.g., consume fewer entries as data in Aqueduct is better packed, or by coalescing entries when contiguous). Overall, Aqueduct's memory organization greatly enhances the efficiency of bulk slot transfers critical for rapid elasticity.

B. Applicability and design goals

Aqueduct's fault-tolerant migration algorithm targets in-memory key-value stores. We assume that the KVS (i) partition the key space into logical units such as shards or slots, (ii) replicate each partition across a replication group using a leader-based replication protocol (e.g., Raft) with synchronous writes, and (iii) operate under a fail-stop failure model where nodes may crash and later rejoin.

Overall Aqueduct's fault-tolerant migration protocol described in the following sections assumes a replicated KVS that uses Raft for consistency.

C. Fault Model

Aqueduct operates under a *fail-stop* failure model, in which nodes may crash and permanently stop executing. A failed

[1]RDMA operations require that memory regions (MRs) be *registered* with RNICs, assigning them Remote Keys (RKeys) used in remote access

node does not recover unless explicitly restarted and reintroduced into the system.

The system employs synchronous replication for write operations using the Raft consensus protocol. Data is organized into fixed-size replication groups, each consisting of a leader and a set of followers. A write is considered committed only after it has been durably replicated to a majority of the nodes in its replication group.

Aqueduct assumes that at least one replica holding committed data remains alive in each replication group. Under this assumption, the system guarantees durability and consistency of committed updates. If all replicas in a replication group crash simultaneously, the data hosted by that group is permanently lost.

Elasticity operations involve two replication groups. A *donor* group that currently owns the data being migrated, and a *recipient* group that will assume ownership. Migration proceeds concurrently with client requests. The migration protocol tolerates crashes according to the Raft failure model and guarantees that no committed data is lost as long as the above majority assumption holds.

Failures during migration do not require restarting the entire operation. Instead of rolling back a partially completed migration transaction, the protocol always makes forward progress: once a group participates in a migration, it will eventually be transferred to its recipient group, along with all other groups in the transaction.

D. Fault Tolerant Migration protocol

The migration protocol is designed to operate concurrently with client requests and relies on Raft for coordination of source and reciepient, and one-sided RDMA for efficient bulk data transfer. Data migrations in Aqueduct are managed by the *Migration Manager (MM)* module, which orchestrates the entire migration process on the source and destination nodes. The MM ensures seamless data transfer and coordinates the ownership handover upon successful migration. The migration process involves both the transfers of data and its integration into the destination node's active dataset.

During migration, there are two replica groups involved. the *donor* (source) group, which holds the data to be migrated, and the *recipient* (destination) group, which will receive and integrate the migrated data. Each group has a leader and followers. The leader coordinates the migration process within its group.

At first, the protocol should transfer the data from the source to the destination (leader-to-leader) and ensure the migrating data can tolerate crashes before stoping the double reads. Before stopping the double reads and disconnecting the source replica group for that group, the destination replica group should transfer the data to a majority of replicas (leader-to-replica) and integrate the received data into leader recipient recplica's active dataset via index updates (Section I-G). Leader-to-replica on the destination replica group, have two paths. The fast path which the majority of the replicas have received the migrating data. This path must finish before stopping the double reads to ensure that the data is available on a majority of replicas. The slow path which the minority of the replicas have not received the data. This can happen asynchronously and can complete after stopping the double reads. At some point all nodes will eventually receive the migrated data from either the fast path or the slow path. Fig. 2 provides an overview of Aqueduct's migration process. Migration is initiated by a coordinator C, which instructs target source replica leader to start a transaction and migrate a set of slots to a destination replica leader D. The protocol proceeds in subtransactions (G_i), each corresponding to a subset of the total slots. For each G_i, source replica group records the start of G_i. For each group, the source first enters a prepare phase, during which it establishes RDMA buffers and exposes memory regions containing the data to be migrated. Buffer metadata, including RDMA pointers and access keys, are disseminated to the destination group via a reliable control path.

Once preparation completes, the source leader performs one-sided RDMA WRITE operations to write data directly from the source leader replica to the destination side replica leader. Upon a transfer completion of a group G_i, the source leader issues an RDMA_DONE acknowledgment.

To ensure correctness under failures, the protocol requires a majority acknowledgment within the destination replication group before marking a batch as successfully transferred. Since the majority of replicas in the destination group must confirm that received the data, the protocol guarantees that at least one replica will have the complete data even in the event of failures, therefore clients can safely ask and proceed to the next group. At this point, the group is marked as completed (Gi Done) and committed via Raft on source replication group.

During migration of a group, writes are directed to the destination replica group, while reads may be served from either the source or destination group, depending on the migration state of the accessed data. This guarantees that no committed updates are lost.

After all groups complete, a transaction is considered complete and can safely cleanup the transaction state.

E. Migration Phases

The migration of a slot group G_i proceeds in a sequence of phases coordinated by the leaders of the donor and recipient replication groups. These phases are designed to enable fast data transfers while preserving correctness under failures.

Prepare phase The migration manager on the donor leader initiates the migration of group G_i by entering a prepare phase. During this phase, the donor leader identifies the memory blocks associated with the slots in G_i and exposes them as RDMA-accessible memory regions. The donor disseminates the corresponding buffer metadata, including memory addresses, sizes, and RDMA access keys, to the recipient group through a reliable control path. This phase establishes all necessary state for bulk data transfer without moving any data.

Bulk transfer phase Once preparation completes, the donor leader performs one-sided RDMA `WRITE` operations to transfer the data of group G_i directly into pre-allocated memory on the recipient leader. Data movement occurs entirely off the CPU fast path and does not require involvement of the recipient leader during transfer. After all RDMA operations for G_i complete, the donor leader issues an explicit `RDMA_DONE` notification to the recipient group.

Replication on the recipient Upon receiving the transferred data, the recipient leader propagates the migrated state to the replicas in its replication group. This replication follows two paths. In the *fast path*, the recipient leader synchronously ensures that the migrated data is durably replicated to a majority of replicas. This step is required before the system can safely stop serving reads from the donor group. In the *slow path*, the remaining replicas receive the migrated data asynchronously after the group is made available for serving. Eventually, all replicas converge to the same state.

Commit and group completion After a majority of the recipient replicas acknowledge successful replication of group G_i, the migration of the group is considered complete. The donor leader records the completion of G_i as a committed event via Raft, making the progress durable. At this point, ownership of the slots in G_i is transferred to the recipient group, and the system proceeds to the next migration group. This design guarantees that, even in the presence of failures, at least one replica in the recipient group holds the complete migrated data.

F. Serving Client Requests During Migration

Aqueduct is designed to migrate data without interrupting client-facing operations.

For slots that are not currently under migration, requests are handled normally by their owning replication group. For slots belonging to an active migration group G_i, request routing depends on the operation type and the migration phase.

Writes Once migration of a group G_i begins, all new write operations targeting slots in G_i are redirected to the recipient replication group. Writes are processed through the recipient leader and synchronously replicated using Raft, ensuring that all committed updates are preserved even if failures occur during migration. This redirection prevents divergence between donor and recipient replicas and eliminates the need for write-back or reconciliation after migration.

Reads Read requests targeting slots in G_i may be served by either the donor or the recipient group, depending on data availability and migration progress. Before the completion of the fast replication path on the recipient side, reads may be served from the donor group to avoid exposing incomplete state. After a majority of recipient replicas have received the migrated data, reads are served exclusively by the recipient group. This controlled transition ensures that reads never observe partially migrated or unreplicated state.

By redirecting writes to the recipient group and carefully transitioning read ownership only after majority replication, Aqueduct guarantees that no committed updates are lost and that clients observe a consistent view of the data. At any point during migration, each key is served by at least one replica group that contains a complete and up-to-date version of the data.

G. Index updating

Transferring data using RDMA enables the source node to write data directly into the target node's memory. However, transferring raw data alone is insufficient. In Aqueduct, a dictionary-like data structure serves as an index, maintaining pointers to the actual key-value pairs. Therefore, after the data transfer completes, the dictionary is updated to reflect the new memory locations—a process referred to as the *index update*.

Aqueduct handles index updates using dedicated background threads with lower scheduling priority to minimize interference with client-serving operations. Since these background threads run concurrently with client threads, multiple threads may access and update the dictionary simultaneously. To ensure consistency, Aqueduct employs fine-grained locking for dictionary access and modification.

During migration, all new writes are directed to the destination node. Consequently, data received from the source may contain stale versions of keys. As part of the index update, Aqueduct detects and discards these stale entries, ensuring that only the most recent values are preserved in the dictionary.

Algorithm 1 MIGRATE_GROUP (Leader-to-Leader)

```
procedure MIGRATE_GROUP(G_i, L_r)
    RAFT_APPEND(G_i_START)
    RAFT_COMMIT()
// get the block addresses for the slots
    B_s ← ALLOCATOR.get_blocks(G_i.slots)
// flush the logged writes to the actual blocks
    for each slot in G_i.slots do
        FLUSH_LOG_ENTRIES_TO_BLOCK(slot)
    end for
// register the RDMA memory regions
    M_s ← REGISTER_RDMA(B_s)
// send prepare RPC to recipient with buffer metadata
    SEND(L_r, PREPARE(G_i, meta(M_s)))
    M_d ← WAIT(L_r, PREPARE_OK)
// perform RDMA transfer
    for (m_s, m_d) in ZIP(M_s, M_d) do
        RDMA_WRITE(m_s.addr, m_d.addr, m_s.size, m_d.rkey)
    end for
// wait for all RDMA writes to complete
    WAIT_RDMA(G_i)
// notify recipient that RDMA is done
    SEND(L_r, RDMA_DONE(G_i))
// wait for majority replication on recipient
    WAIT(L_r, G_i_DONE(G_i))
// mark migration group as done in Raft
    RAFT_APPEND(G_i_DONE)
    RAFT_COMMIT()
end procedure
```

Algorithm 2 Recipient Leader Handling of Migrated Data

```
upon RECEIVE PREPARE(G_i, metadata) from donor_leader do
buffers ← ALLOCATE_BUFFERS(G_i, metadata)
REGISTER_RDMA(buffers)
STORE_METADATA(G_i, metadata, buffers)
SEND_RPC(donor_leader, PREPARE_OK(G_i))
end upon

upon RECEIVE RDMA_DONE(G_i) from donor_leader do
RDMA_RECEIVED[self][G_i] ← true
// Build replication chain
alive_replicas ← []
for each replica r in REPLICA_SET(G_i) do
    if CHECK_ALIVE(r) then
        alive_replicas.append(r)
    end if
end for
chain ← BUILD_CHAIN(alive_replicas)
REPLICATE_TO(r, G_i)
WAIT UNTIL MAJORITY OF (RDMA_RECEIVED[R_i][G_i] = true)
RAFT_APPEND_LOG(G_i_INDEX_UPDATE)
RAFT_COMMIT()
EXECUTE_UPDATE_INDEX(G_i)
// Notify donor leader
SEND_RPC(donor_leader, G_i_Done(G_i))
end upon

upon APPEND_ENTRIES(entry) from R_i do
if entry.metadata = RDMA_DONE(txn_id, G_i) then
    RDMA_RECEIVED[R_i][G_i] = true
end if
APPEND_TO_LOG(entry)
REPLY_SUCCESS()
end upon
```

Algorithm 3 Donor Leader Recovery

```
upon LEADER_ELECTION_COMPLETE do
if NOT (self.group_type is DONOR and self is LEADER) then
    return                                  ▷ Only new donor leader recovers
end if
// Check if there is any active migration txn
if ¬ HAS_ACTIVE_MIGRATION_TXN() then
    return
end if
// Load migration progress from Raft
started_groups ← RAFT_QUERY_Gi_WITH_STATE(STARTED)
done_groups ← RAFT_QUERY_Gi_WITH_STATE(DONE)
for each G_i in started_groups do
    if G_i ∈ done_groups then
// Already safely completed before failure
        MARK_GROUP_COMPLETE(G_i)
        continue
    end if
// Query recipient leader for current G_i state
    SEND_RPC(recipient_leader, QUERY_Gi_STATE(G_i))
    state ← WAIT_REPLY(recipient_leader)
    if state = Gi_Done then
// Recipient has quorum durability → finalize locally
        RAFT_APPEND_LOG(Gi_DONE)
        RAFT_COMMIT()
    else if state = TRANSFERRED then
// RDMA completed, replication still ongoing
        WAIT_FOR_Gi_DONE(G_i)
    else if state = PREPARED then
// Buffers exist but no data → re-issue RDMA WRITEs
        START_RDMA_TRANSFER(G_i)
        WAIT_FOR_COMPLETION(G_i)
    else
// Unknown or transient state → skip and revisit later
        DEFER_Gi(G_i)
    end if
end for
// Global txn completion check
if ALL_GROUPS_DONE() then
    RAFT_APPEND_LOG(TXN_MIGRATION_DONE)
    RAFT_COMMIT()
end if
end upon
```

Algorithm 4 Recipient Leader Recovery

```
upon LEADER_ELECTION_COMPLETE do
if self.group_type ≠ RECIPIENT or self ≠ LEADER then
    return
end if
// Groups for which this recipient has metadata or data but INDEX_UPDATE not yet committed
active_groups ← QUERY_Gi_WITH_STATE(PREPARED) OR WITH_STATE(TRANSFERRED)
for each G_i in active_groups do
    WAIT_UNTIL_MAJORITY_OF_RDMA_RECEIVED[Ri][Gi] = true
    if RDMA_RECEIVED[self][Gi] == false then
        ASK_THE_DATA_FROM_DONOR
        WAIT_UNTIL_RDMA_DONE_OF_Gi
    end if
    RAFT_APPEND(INDEX_UPDATE(G_i))
    RAFT_COMMIT()
    APPLY_UPDATE_INDEX(G_i)
end for
end upon
```

Algorithm 5 Recipient Node Recovery

upon REPLICATION_FAILURE(G_i, R_{i+1}) **do**
MARK_NODE_FAILED(R_{i+1})
next $\leftarrow$ FIND_NEXT_ALIVE_NODE($i+1$)
UPDATE_CHAIN(G_i, **replace** R_{i+1} **with** next)
ASYNC_REPLICATE_TO(next, G_i)
end upon

Q: When is a group of slots G considered reliably transferred to a group of Raft replicas?

A: When a majority of Raft replicas have successfully received the RDMA transfers for that group, and have recorded in their log the need to update their index with those keys before responding to a get(Key). At least one of these replicas will survive to carry G in the future and is guaranteed to remember to update index (as it has been committed into its log).

A replica votes to accept an UPDATE_INDEX(txn_id, G) command only when it knows to have received the RDMA transfers for (txn_id, G).

Committing the UPDATE_INDEX(txn_id, G) command in a Raft majority that has received the RDMA buffers for G is needed to ensure that at least one replica that can be elected leader has received the RDMA buffers (and thus can further propagate them as part of replica-sync actions in the future).

Invariant: A replica that has the UPDATE_INDEX(txn_id, G) command in their log, also has the RDMA buffers that come with it. If that log entry is also committed, a majority of replicas have the RDMA buffers and remember to execute update-index on them.

Q: When does a leader know when to propose an update-index command?

A: It must have received feedback through replicas (via replies to the periodic AppendEntries RPC) about whether they have received the RDMA buffers for (txnid, G) or not.

Q: How does the chain replication protocol for RDMA transfers work?

A: The leader proposes a chain (order of replicas) to be followed, and kick-starts RDMAs to the next replica, notifying it when done (RDONE). An RDONE is the trigger for the next node to do the same. Nodes respond about the status of RDMA buffers to AppendEntries.

Q: When is it safe to stop double reads for a group?

A: When the group has been reliably transferred to the recipient group. In that case, it is guaranteed that the recipient group will authoritatively serve G (each replica is guaranteed to have done update-index and all key updates committed, prior to serving a get(K)).

Thus the leader of a recipient group can respond (txn_id, G)-OK to a RDONE(txn_id, G) RPC from the donor leader when it has committed update-index(txn_id, G).

Q: What happens to a minority that has not yet received the RDMA transfers for G, even though an UPDATE_INDEX(txn_id, G) command has been committed?

A: They either receive it later as part of an ongoing chain of RDMAs or they receive it from the leader as part of a log-sync action (when executing AppendEntries and one of the entries is UPDATE_INDEX(txn_id, G)).

Q: Why is it necessary for a leader replica to add a tombstone in their index when deleting a key K belonging to G, prior to executing an update-index operation for G?

A: Imagine that a recipient leader L receives an update for key K, then a delete for K, and then it runs update-index bringing K into the index. If you do not leave a tombstone for K, an old version of K is going to be resurrected, breaking consistency.

Q: Does a replica need to know that a migration transaction is happening?

A: Yes, so that it can handle index updates differently (tombstones)! A txn_start(G) command needs to be committed prior to executing operations for G.

Each replica also needs to know certain information about the transfer (txn_id, G, etc.). The chain of replicas to be followed could be an optimization - the leader may propose this to be the replicas that have responded positively to the txn_start(txn_id, G) command.

References

[1] "Mongodb," https://www.mongodb.com, accessed: 2025-06-12.

[2] "Redis," https://redis.io/, accessed 2025-05-04.

[3] A. Lakshman and P. Malik, "Cassandra: a decentralized structured storage system," *SIGOPS Oper. Syst. Rev.*, vol. 44, no. 2, p. 35–40, Apr. 2010. [Online]. Available: https://doi.org/10.1145/1773912.1773922